

Plan Séquentiel - Extension du Backend Orchestrateur Zarr Conversion + Masking (OmniCloudMask / OmniWaterMask)

Contexte

Le backend orchestrateur doit être étendu avec une logique microservices:

- fetcher-service: télécharge les scènes brutes
- zarr-converter-service: convertit les scènes en Zarr
- mask-service: applique les masques cloud/eau sur le Zarr

Règle de travail

- Les tâches sont séquentielles: on ne passe pas à la suivante avant validation de la précédente
- La taille des tâches est moyenne: ni trop courte, ni trop longue
- A = Mehdi
- B = Ksmohamm

Pipeline Cible

1. Fetch du produit brut
2. Conversion en Zarr
3. Application des masques cloud/eau
4. Stockage des artefacts et métadonnées
5. Job termine avec statut complet

Backlog Séquentiel

1. Owner: A
Tâche: Définir l'architecture cible et les contrats inter-services
Couvre:
 - responsabilités de fetcher-service, zarr-converter-service, mask-service
 - payloads d'entrée/sortie
 - états du pipelineCritère de fin:
 - un document simple valide par les deux avec les contrats et les états
2. Owner: B
Tâche: Créer le squelette du zarr-converter-service
Couvre:
 - structure du projet
 - dépendances Python
 - conteneur
 - point d'entrée de serviceCritère de fin:
 - le service démarre en local avec un endpoint ou worker vide mais fonctionne
3. Owner: A
Tâche: Définir le modèle Zarr standard du projet
Couvre:
 - layout (imagery, masks/cloud, masks/water, metadata)
 - dimensions
 - chunks
 - compression
 - métadonnées obligatoiresCritère de fin:
 - une spec Zarr claire et partagée
4. Owner: B
Tâche: Implémenter la lecture et la normalisation Sentinel-2 (.SAFE)
Couvre:
 - identification des bandes utiles
 - lecture des données
 - mapping vers un schéma de bandes standard

- Critere de fin:
- une scene Sentinel-2 est lue et convertie en representation interne normalisee
5. Owner: A
Tache: Implementer la lecture et la normalisation Landsat
Couvre:
- identification des bandes utiles
- lecture des donnees
- mapping vers le meme schema standard que Sentinel-2
Critere de fin:
- une scene Landsat est lue et convertie en representation interne normalisee
6. Owner: B
Tache: Implementer l'écriture Zarr
Couvre:
- écriture de imagery
- écriture de metadata
- compatibilite Sentinel-2 et Landsat
Critere de fin:
- un Zarr valide est genere et relisible pour les deux providers
7. Owner: A
Tache: Ajouter la robustesse et les tests du convertisseur
Couvre:
- gestion des bandes manquantes
- gestion des scenes corrompues
- manifest
- checksum
- tests unitaires et d'integration
Critere de fin:
- le convertisseur echoue proprement et passe ses tests
8. Owner: B
Tache: Creer le squelette du mask-service et integrer les dependances Omni*
Couvre:
- structure du projet
- conteneur
- installation de OmniCloudMask
- installation de OmniWaterMask
Critere de fin:
- le service demarre avec les dependances chargees
9. Owner: A
Tache: Integrer OmniCloudMask sur le Zarr standard
Couvre:
- lecture des bandes requises depuis le Zarr
- execution du modele
- generation d'un cloud mask exploitable
Critere de fin:
- un masque cloud est genere a partir d'un Zarr test
10. Owner: B
Tache: Integrer OmniWaterMask sur le Zarr standard
Couvre:
- lecture des bandes requises depuis le Zarr
- gestion de la contrainte reseau / OSM si necessaire
- generation d'un water mask exploitable
Critere de fin:
- un masque eau est genere a partir d'un Zarr test
11. Owner: A

Tache: Ecrire les masques dans le Zarr et ajouter les statistiques

Couvre:

- ecriture dans masks/cloud
- ecriture dans masks/water
- pourcentage cloud
- pourcentage water
- pourcentage nodata

Critere de fin:

- le Zarr final contient les masques et leurs stats

12. Owner: B

Tache: Ajouter la robustesse et les tests du mask-service

Couvre:

- gestion des bandes absentes
- timeouts
- erreurs de runtime Omni*
- tests unitaires et d'integration

Critere de fin:

- le mask-service echoue proprement et passe ses tests

13. Owner: A

Tache: Brancher le zarr-convert-service dans l'orchestrateur

Couvre:

- ajout du job convert_to_zarr
- ajout du payload
- appel inter-service
- persistance de zarr_uri

Critere de fin:

- l'orchestrateur lance la conversion Zarr apres le fetch

14. Owner: B

Tache: Brancher le mask-service dans l'orchestrateur

Couvre:

- ajout du job apply_masks
- ajout du payload
- appel inter-service
- persistance de mask_summary

Critere de fin:

- l'orchestrateur lance le masquage apres la conversion

15. Owner: A

Tache: Chainer le pipeline complet et gerer les retries par etape

Couvre:

- fetch -> convert_to_zarr -> apply_masks
- retries separees
- eviter le re-telechargement inutile
- gestion des artefacts finaux

Critere de fin:

- le pipeline complet s'execute en bout en bout sur au moins un cas

16. Owner: B

Tache: Validation finale et packaging deployment

Couvre:

- test end-to-end Sentinel-2
- test end-to-end Landsat
- images conteneur finales
- verification compose / Kubernetes

Critere de fin:

- pipeline valide et pret a etre deployee

Blocs de Travail

- Bloc 1: Conversion Zarr = taches 1 a 7

- Bloc 2: Masking = taches 8 a 12
- Bloc 3: Intégration Orchestrateur = taches 13 a 16

Points Critiques

1. Figurer les contrats avant tout développement
2. Produire un Zarr stable et lisible
3. Valider le comportement de OmniWaterMask (notamment la partie reseau / OSM)
4. Isoler les retries par etape pour eviter de refaire inutilement le fetch

Definition de Reussite

- Une scene est telechargee par le fetcher
- Le Zarr est produit par le zarr-converter-service
- Les masques cloud/eau sont ajoutes par le mask-service
- L'orchestrateur connait chaque etape, chaque artefact, et chaque statut
- Le pipeline complet tourne de bout en bout sur Sentinel-2 puis Landsat